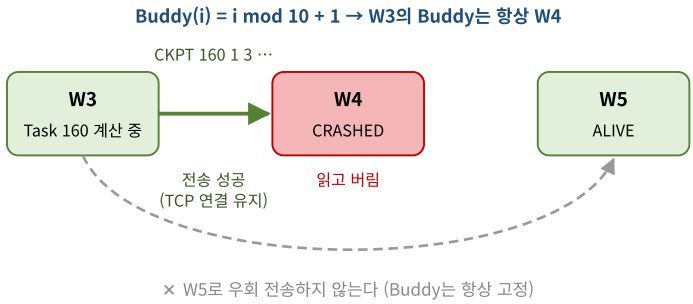
HW#3 Q&A — Checkpoint · Takeover · Epoch · Owner 쉽게 이해하기

학생 질문 6가지를 그림과 예제로 정리

(규칙의 근거 — 과제설명 PPTX: p.21 Heartbeat·FD · p.22 Checkpoint · p.23 Takeover · p.24 Commit·Epoch · p.25 Rejoin · p.31 Buddy가 두 Task 처리 · p.33 공유 자원 · p.35 고아 Task · p.38–39 메시지 처리 규칙 · p.41–42 의사코드 · p.47 S5 이웃 동시 crash)

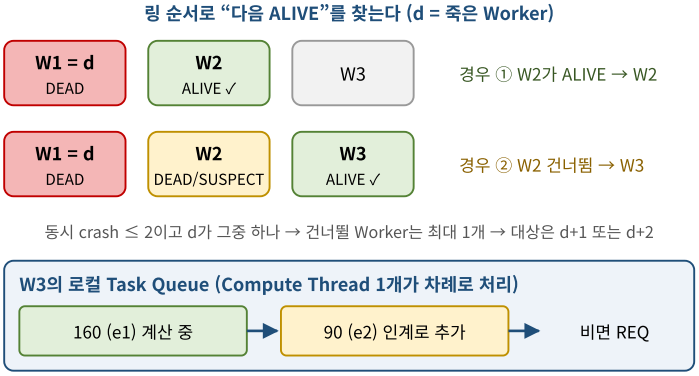
※ 각 Q의 예는 **서로 독립된 가상 상황**이다(Q5만 seed=42 예). 예를 들어 Q1의 W4 crash와 Q2의 W1 · W2 crash는 같은 시점의 일이 아니다. 시각은 설명용 예시이며, 소수 개수는 실제 값이다.

Q1. W3가 CKPT를 Buddy W4에게 보내는데 W4가 죽어 있으면? W5에게 대신 보내나?



- **W3는 W4가 죽은 것을 모른다.** 가상 crash는 TCP 연결을 끊지 않으므로 전송이 오류 없이 성공하고, CRASHED 상태의 W4는 받은 CKPT를 읽고 버린다.
- **알 필요도 없다.** 살았는지 죽었는지 판단은 Coordinator만 한다(HB 기반). Worker끼리는 판단하지 않는다.
- **W5로 보내지 않는다.** 규칙: “CKPT는 항상 고정 Buddy에게, Buddy가 crash 중이면 버려진다. 보내는 쪽은 신경 쓰지 않는다.”
- **그래도 결과는 정확하다.** CKPT는 속도를 위한 보험일 뿐이다. 이때 W3까지 crash하면 W3의 Task는 다음 ALIVE(W4가 DEAD이므로 W5)에게 넘어가고, W5에는 CKPT가 없으므로 **step 1부터** 다시 계산한다. 느려질 뿐 값은 같다.
- W4가 재합류하면 Store는 빈 상태로 시작하고, W3의 다음 CKPT부터 다시 쌓인다.

Q2. Coordinator는 crash한 Worker를 어떻게 관리하고, 그 Task를 누구에게 다시 맡기나?



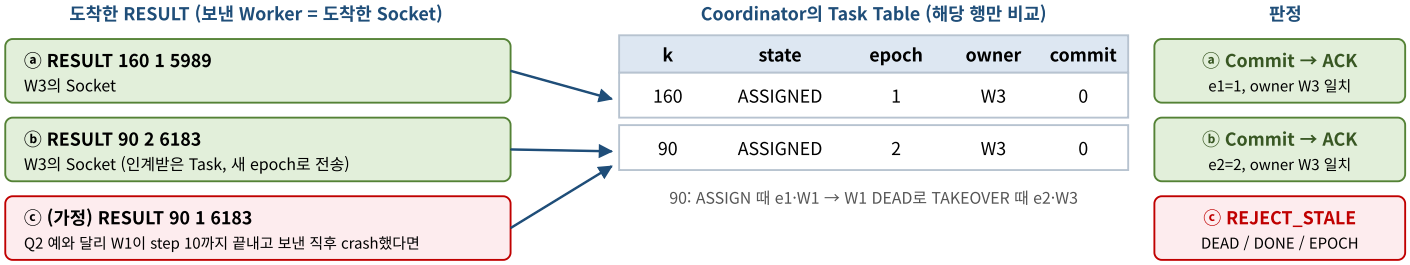
#	처리 순서
①	**감지:** Coordinator는 Launcher에게 듣지 않는다. 각 Worker의 마지막 HB 시각(lastHB)만 보고 Worker Table에 ALIVE / SUSPECT(3초↑) / DEAD(5초↑)를 유지한다.
②	**대상 선택:** d+1, d+2, …에서 첫 **ALIVE** b (DEAD · SUSPECT 모두 건너뜀)
③	**넘기는 Task:** owner = d인 **모든** ASSIGNED Task (d가 인계받았던 Task 포함). Task마다 epoch + 1, owner = b → `TAKEOVER k e from=d`
④	**b의 처리:** 로컬 Queue에 한 줄 추가(동시 계산이 아니라 차례로). Store에 CKPT가 있으면 그다음 step부터, 없으면 step 1부터. 권장 순서: 진행 중 Task 먼저
⑤	**반납:** Task마다 `RESULT k e v`를 각각 보낸다. REQ는 Queue가 **완전히 빈 뒤**에만
⑥	**예외:** b도 crash하면 b가 가진 Task(인계받은 것 포함)가 다시 다음 ALIVE로, epoch 한 번 더 +1. d가 재합류하면 옛 Task는 돌려받지 않고 새 Task를 REQ로 받는다

예제 (가상) — W1의 Task 90이 W3에게 가는 과정 (Q3로 이어짐)

순서	사건	Coordinator · Worker의 상태
①	W1이 Task 90(e1)을 step 4까지 계산(CKPT는 Buddy W2의 Store에)한 뒤 crash	Task[90] = (e1, owner W1). Coordinator는 아직 모름 (HB만 끊김)
②	약 1.5초 뒤 W2도 crash → W2의 Store(90의 CKPT 포함)도 사라짐	동시 crash 2개 (W1, W2)
③	W1이 DEAD 판정(HB 5초↑). 이 순간 W2는 HB가 끊긴 지 3~5초라 **SUSPECT** → 건너뜀 → **W3**	Task[90] = (**e2**, owner **W3**) → `TAKEOVER 90 2 from=1`
④	W3: 진행 중이던 160(e1)을 마저 계산, 90은 Queue에서 대기. W3의 Store에는 90의 CKPT가 없음	W3 Queue = [(90, e2, step 1, 0)] → 160을 끝낸 뒤 90을 **step 1부터** 계산
⑤	W3: `RESULT 160 1 5989`, 이어서 `RESULT 90 2 6183`	두 RESULT를 각각 자기 Task 행과 비교 → 둘 다 Commit (Q3 그림)
⑥	W3의 Queue가 비었음 → `REQ 3`	다음 PENDING Task를 epoch 1로 배정

Q3. W3가 Task 160(배정)과 Task 90(인계)을 함께 가지면 epoch가 서로 다른데, Coordinator는 무엇을 보고 Commit하나?

핵심: epoch는 Worker의 번호가 아니라 Task마다 붙는 번호다. Coordinator는 RESULT의 Task 번호 k로 Task Table[k] **한 행만** 찾아 그 행의 (epoch, owner)와 비교한다. 다른 Task의 epoch는 보지 않는다. 그래서 한 Worker가 epoch가 다른 RESULT를 보내는 것은 정상이다.



ⓒ의 거부 이유 (판정 순서대로 첫 번째로 걸리는 것):
W1이 아직 DEAD → DEAD · W1이 재합류했고 90이 이미 Commit됨 → DONE · 아직 Commit 전 → e1 ≠ 2이고, 보낸 W1 ≠ owner W3 → EPOCH. 어느 경우든 +0

RESULT k e v 판정 순서 (Coordinator, 모든 조 동일)

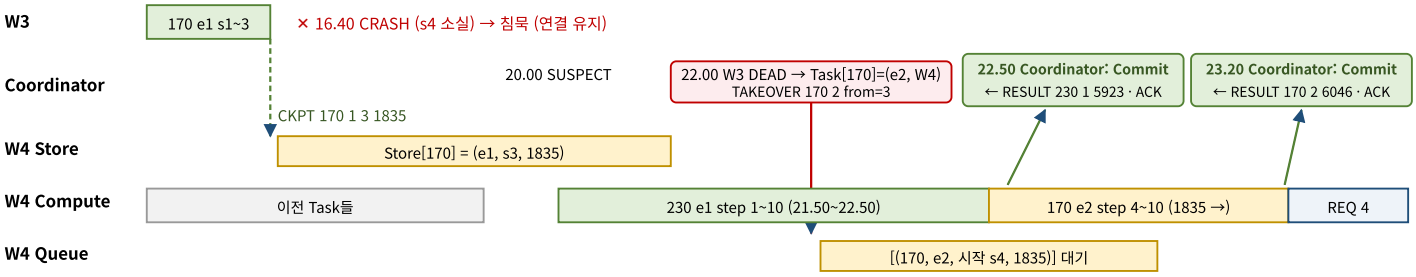
#	조건	결과
①	보낸 Worker(도착한 Socket)가 DEAD	REJECT_STALE DEAD
②	Task[k]가 이미 DONE	REJECT_STALE DONE
③	e ≠ Task[k].epoch 또는 보낸 Worker ≠ Task[k].owner	REJECT_STALE EPOCH
④	그 외	**Commit**: DONE, result[k]=v, commit_count+1 → ACK k

②~④는 Task[k]의 Lock 안에서 **확인과 변경을 한 번에** 한다. Lock 밖에서 확인하고 나중에 바꾸면, 그 사이 인계가 끼어 commit_count = 2(이중 Commit)가 될 수 있다.

epoch가 바뀌는 때와 Worker가 쓰는 값

사건	Task[k]의 변화 / Worker 동작
ASSIGN (REQ 응답)	epoch = 1, owner = 요청자
TAKEOVER (인계)	epoch + 1, owner = 다음 ALIVE. 여러 번 인계되면 e1 → e2 → e3 …
Worker의 저장	로컬 Queue에 (k, e, 시작 step, partial)로 저장
CKPT · RESULT	그 Task에 받은 e를 **그대로** 사용
CKPT 저장(Buddy)	(epoch, step) 사전식 비교로 최신만 보관. 인계 시에는 어느 epoch의 CKPT든 이어서 사용 가능(소수 개수는 누가 세어도 같음)

Q4. W4가 W3의 Task 170(CKPT s3까지)을 이어받아 자기 Task 230과 함께 끝내고 반납하는 과정은? (비슷한 예: PPTX p.31)



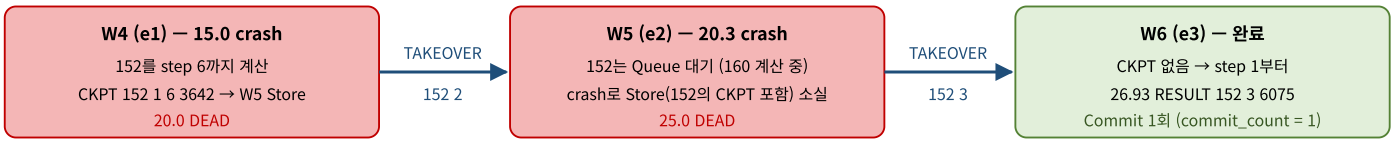
CKPT 전송: 230(e1)과 170(e2) 모두 W4의 Buddy인 W5에게 · 시각은 설명용 예시 · 소수 개수는 실제 값

시각	사건
16.30	W3: 170 step 3 완료 → `CKPT 170 1 3 1835` → W4 Store[170] = (e1, s3, 1835)
16.40	W3 CRASH (마지막 HB 16.35). Coordinator는 모름
22.00	W3 DEAD → 다음 ALIVE = W4 → Task[170] e1→**e2**, owner W4 → `TAKEOVER 170 2 from=3`. W4는 230 step 6 계산 중
22.01	W4 Control Recv: Store[170] 있음 → Queue에 (170, e2, 시작 step **4**, partial **1835**). Queue가 비어 있지 않으므로 REQ 안 보냄
22.50	`RESULT 230 1 5923` → Task[230] (e1, W4) 일치 → Commit, ACK
23.20	170 step 4~10: 1835 → 2432 → … → 6046 → `RESULT 170 2 6046` → Task[170] (e2, W4) 일치 → Commit, ACK → Queue 비었으므로 `REQ 4`

- **동시 계산이 아니다:** 230을 끝낸 뒤 170 (권장). 170을 먼저 하려면 진행 중이던 230을 (230, e1, 시작 step 6, partial 2998)로 Queue에 다시 넣는다
- **이어 세기:** W4가 W3의 Buddy라 CKPT가 있다 → step 4부터. CKPT의 epoch(1)와 새 epoch(2)가 달라도 사용
- **Task별 epoch:** 230은 e1, 170은 e2로 각각 반납 → 각자 행과 비교해 한 번씩 Commit
- **백업 경로 변경:** 170의 CKPT는 이제 W5로 → W4까지 crash해도 W5가 e3로 이어받음
- **지표:** 감지 지연 5.65초 · 복구 1.2초 · CKPT 절약 3/10

Q5. 인계받은 Worker까지 crash하면? — 한 Task가 두 번 인계되는 경우 (Task 152, seed=42 예 · PPTX p.47 S5)

Task 152의 owner와 epoch 변화: W4(e1) → W5(e2) → W6(e3)



- 25.0에 W5가 DEAD일 때 Coordinator는 W5가 owner인 **모든** Task(자기 160 + 인계받은 152)를 W6에게 넘긴다 → 152를 빠뜨리면 영원히 미완료(999/1000에서 멈춤)
- W4의 CKPT는 W5에 있었으나 W5와 함께 사라짐 → W6은 처음부터 계산(느려질 뿐 값은 같음). e1 · e2로 늦게 온 RESULT는 모두 거부

시각	사건	Task[152] (epoch, owner)	들어올 수 있는 RESULT의 운명
14.4	W4가 REQ → `ASSIGN 152 1`	(1, W4)	RESULT 152 1 from W4 → Commit 가능
15.0	W4 crash (step 6 CKPT는 W5에)	(1, W4) — Coordinator는 모름	—
20.0	W4 DEAD → 다음 ALIVE W5 → `TAKEOVER 152 2 from=4`	(2, W5)	e1 결과는 이제 거부
20.3	W5도 crash (152는 Queue에서 대기 중이었음)	(2, W5) — 모름	—
25.0	W5 DEAD → 다음 ALIVE W6 → `TAKEOVER 152 3 from=5`	(3, W6)	e1 · e2 결과는 모두 거부
26.93	W6: CKPT 없음(W5와 함께 소실) → step 1부터 → `RESULT 152 3 6075`	DONE, commit_count = 1	**Commit** → 이후 같은 결과는 DONE으로 거부

Q5-1. epoch는 누가 관리하고 올리나? — Coordinator만 (Worker는 받은 값을 그대로 사용)

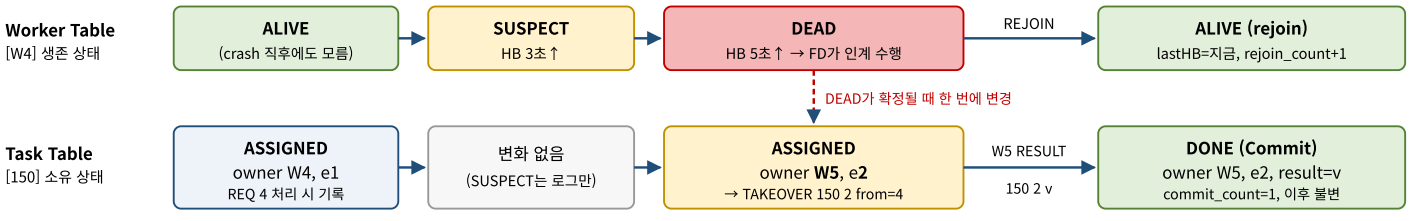
시점	바꾸는 주체	Task[152]	Worker에게 알리는 메시지
14.4 W4의 REQ	Dispatcher	e = **1**, owner W4	`ASSIGN 152 1` → W4
20.0 W4 DEAD	Failure Detector (Task Lock 안)	e 1 → **2**, owner W5	`TAKEOVER 152 2 from=4` → W5
25.0 W5 DEAD	Failure Detector (Task Lock 안)	e 2 → **3**, owner W6	`TAKEOVER 152 3 from=5` → W6

- **epoch의 원본은 Coordinator의 Task Table**. 바뀌는 때는 ASSIGN(=1)과 TAKEOVER(+1) 두 가지뿐이고, Lock 안에서 바꾼 뒤 Lock 밖에서 메시지로 알린다.
- **Worker는 계산하지 않는다.** W6는 받은 3을 Queue에 (152, e=3, …)로 저장하고 CKPT · RESULT에 그대로 쓴다. "인계받았으니 +1" 하면 안 된다.
- **Worker는 이력을 모른다.** W5는 152를 계산하기 전에 crash해 e2로 보낸 메시지가 없다. W6가 올바른 번호(3)를 알 방법은 Coordinator의 통보뿐이다.
- 기준값을 가진 Coordinator가 번호를 매기고 비교하므로, 정당한 담당자의 결과만 Commit된다(Fencing).

Q6. owner도 Coordinator만 관리하나? — Task 150을 W4에게 줄 때와 W4가 crash했을 때의 기록 변화

답: 그렇다. owner · epoch · state는 모두 Coordinator의 **Task Table**에만 있고 Coordinator만 바꾼다. Worker는 owner 값을 따로 갖지 않으며, 자기 로컬 Queue에 Task가 있다는 것만 안다. Coordinator는 RESULT가 **도착한 Socket**으로 보낸 Worker를 알아내 owner와 비교한다.

Coordinator 안의 두 테이블이 시간에 따라 바뀌는 모습 (Task 150, W4)



- rejoin은 **Worker Table만** 바꾼다 → Task[150]은 W5 소유 그대로, W4는 REQ로 **새 Task**를 받는다(옛 Task를 돌려받지 않음)
- W5도 crash하면 Task[150] → owner W6, e3 (같은 방식으로 다시 인계) · 이미 DONE인 Task는 FD가 건너뛴다
- W4가 늦게 보낸 RESULT 150 1 … → W4가 DEAD면 DEAD, 아니면 DONE 또는 EPOCH(owner W5 · e2와 불일치)로 거부

배정 순서 — 기록 먼저, 통보는 나중 (Dispatcher가 REQ 4 처리)

#	동작
①	**Worker Table Lock** 획득 → W4가 DEAD가 아닌지 확인 (ALIVE · SUSPECT면 배정)
②	PENDING Queue에서 맨 앞 번호 150을 꺼냄
③	**Task[150] Lock** 안에서 state = ASSIGNED, **owner = W4**, **epoch = 1** 기록
④	Lock을 모두 푼 뒤 `ASSIGN 150 1`을 W4의 Socket으로 전송

①~③을 한 Lock 구간에서 하는 이유: 확인과 배정이 갈라지면 그 사이 FD가 W4를 DEAD로 처리하고도 150이 W4 소유로 남는 **"고아 Task"**가 생겨 999/1000에서 멈춘다.

Coordinator가 가진 두 테이블

테이블	내용	보호
Worker Table [1..10]	생존 상태(ALIVE/SUSPECT/DEAD), lastHB, rejoin_count, Socket	Lock 1개
Task Table [0..999]	state(PENDING/ASSIGNED/DONE), owner, epoch, result, commit_count	Task마다 Mutex

- 둘을 함께 다룰 때는 항상 **Worker Table Lock → Task Lock**(번호 오름차순)
- 바꾸는 Thread: Dispatcher(REQ · RESULT · REJOIN), Failure Detector(SUSPECT · DEAD · 인계), Listener(lastHB 갱신 · SUSPECT → ALIVE 복귀)
- Worker 쪽에는 이 테이블이 없다. Worker는 로컬 Queue의 (k, e, 시작 step, partial)만 갖는다

시간 순서로 본 기록 변화 (요약)

시점	Worker Table [W4]	Task Table [150]	설명
ASSIGN	ALIVE	ASSIGNED, owner W4, e1	배정
W4 crash 직후	ALIVE (모름)	그대로	HB만 끊김
HB 3초 없음	**SUSPECT**	그대로	로그만, Task는 손대지 않음
HB 5초 없음	**DEAD**	owner **W5**, epoch **2** (ASSIGNED 유지)	FD → `TAKEOVER 150 2 from=4`
(W5도 crash)	—	owner W6, epoch 3	같은 방식으로 다시 인계
W5 RESULT Commit	—	**DONE**, result, commit_count = 1	이후 바뀌지 않음
W4 재합류	**ALIVE**, lastHB, rejoin_count+1	**변화 없음**	W4는 새 Task를 REQ로 받음

구현 체크 포인트 — 이 Q&A에서 꼭 기억할 것

구성요소	해야 할 것	하면 안 되는 것
Worker (보내는 쪽)	CKPT는 항상 고정 Buddy(i mod 10 + 1)에게. 전송 오류는 WARN 후 계속	Buddy 생사 확인 · 다른 Worker로 우회 전송
Worker (Buddy 쪽)	Task별 최신 CKPT 1개 보관((epoch, step) 사전식). crash 중이면 읽고 버림	CKPT partial을 합계에 사용
Coordinator FD	HB로만 판정, DEAD d의 **모든** ASSIGNED Task를 다음 ALIVE에게, Task마다 epoch+1 · owner 변경 후 TAKEOVER로 새 epoch 통보	Launcher 정보 사용 · SUSPECT에게 인계 · 고정 Buddy에게만 인계
인계받은 Worker	Queue에 (k, e, 시작 step, partial) 추가 → 차례로 계산 → 새 epoch로 CKPT · RESULT, Queue가 비면 REQ	두 Task 동시 계산 · epoch를 스스로 계산 · 옛 epoch로 전송 · Queue가 남았는데 REQ
Coordinator 배정 (REQ)	WT Lock 안에서 DEAD 확인 → PENDING에서 k → Task Lock 안에서 owner · epoch=1 기록 → Lock 밖에서 ASSIGN	확인과 배정 분리 · 기록 전에 통보
Coordinator RESULT	Task[k] 한 행의 (epoch, owner)와 비교, 판정 순서 DEAD → DONE → EPOCH → Commit, Task Lock 안에서 한 번에	Worker 단위 epoch 비교 · Lock 밖 확인 후 변경

정리: ① Buddy는 고정이고 CKPT는 보험일 뿐이다(Worker는 Buddy의 생사를 모른다). ② 인계 대상과 시점은 Coordinator만 HB로 정한다(다음 ALIVE). ③ 인계받은 Task는 Queue에 추가되어 차례로 계산되고, Buddy의 CKPT 다음 step부터 이어 센다. ④ 인계받은 Worker까지 crash하면 그 Task는 다시 다음 ALIVE로 넘어가고 epoch가 한 번 더 오른다. ⑤ owner · epoch · state는 Coordinator의 Task Table에만 있고(재합류는 Worker Table만 변경), epoch는 Task별 번호로 Coordinator만 올리고 ASSIGN · TAKEOVER로 알려 주며, Coordinator는 그 Task의 현재 (epoch, owner)와 일치하는 RESULT만 Task Lock 안에서 한 번 Commit한다 → 이중 Commit 0, 합계 5,761,455.